

INDIVIDUAL PROJECT (LSDS)

WEEK 3 REPORT (MapReduce Implementation)

Name - Kuunal Mistry

AppID - 2410130

1) Objective of Week 3:

The objective of Week 3 was to design and execute a MapReduce program to analyze the large-scale automobile dataset stored in HDFS and extract meaningful insights using distributed processing.

2) Problem Statement:

To compute the average car price per brand using Hadoop Streaming MapReduce. This analysis helps understand pricing trends among different automobile brands.

3) Input Dataset:

- Dataset Name: car_price_big.csv
- Size: ~152MB
- Records Processed: 2,500,001
- Stored in:
/user/maria_dev/car_project/

4) MapReduce Design:

Mapper Logic:

The mapper:

- Reads each row
- Extracts **Brand and Price**
- Emits:
Brand → Price

Example output from mapper:

BMW 54000

Audi 52000

Reducer Logic:

The reducer:

- Groups records by brand
- Calculates total price and count
- Computes average:

$$\text{Average Price} = \text{Total Price} / \text{Count}$$

5) Execution Command:

```
hadoop jar /usr/hdp/current/hadoop-mapreduce-client/hadoop-streaming.jar \  
-file mapper.py \  
-file reducer.py \  
-mapper mapper.py \  
-reducer reducer.py \  
-input /user/maria_dev/car_project/car_price_big.csv \  
-output /user/maria_dev/car_project/output_avg_brand
```

6) Job Execution Details:

From Hadoop job counters:

- Map input records: 2,500,001
- Map tasks executed: 2
- Reduce tasks executed: 1
- Reduce output records: 7
- HDFS bytes read: ~160MB
- File split into: 2 blocks (128MB block size)

This confirms successful distributed processing.

7) Final Output:

Audi	51953.42
BMW	54157.11
Ford	51593.25
Honda	52050.28
Mercedes	53191.09
Tesla	53475.55
Toyota	52078.73

8) Result Interpretation:

BMW has the highest average price.

Tesla and Mercedes follow closely.

Ford has the lowest average price among selected brands.

Luxury brands show consistently higher pricing.

Hadoop successfully processed large-scale automobile data efficiently.

9) Conclusion:

The MapReduce program successfully performed distributed aggregation on a 152MB dataset containing 2.5 million records.

This demonstrates how Hadoop enables scalable and efficient analysis of large datasets.

```
[maria_dev@sandbox-hdp ~]$ ^C
[maria_dev@sandbox-hdp ~]$ vi mapper.py
[maria_dev@sandbox-hdp ~]$ vi reducer.py
[maria_dev@sandbox-hdp ~]$ chmod +x mapper.py
[maria_dev@sandbox-hdp ~]$ chmod +x reducer.py
[maria_dev@sandbox-hdp ~]$ hdfs dfs -rm -r /user/maria_dev/car_project/output_avg_brand
```

```
[maria_dev@sandbox-hdp ~]$ vi mapper.py
[maria_dev@sandbox-hdp ~]$ vi reducer.py
[maria_dev@sandbox-hdp ~]$ chmod +x mapper.py
[maria_dev@sandbox-hdp ~]$ chmod +x reducer.py
[maria_dev@sandbox-hdp ~]$ hdfs dfs -rm -r /user/maria_dev/car_project/output_avg_brand
26/02/26 14:32:24 INFO fs.TrashPolicyDefault: Moved: 'hdfs://sandbox-hdp.hortonworks.com:8020
/user/maria_dev/car_project/output_avg_brand' to trash at: hdfs://sandbox-hdp.hortonworks.com
:8020/user/maria_dev/.Trash/Current/user/maria_dev/car_project/output_avg_brand1772116344945
[maria_dev@sandbox-hdp ~]$ hadoop jar /usr/hdp/current/hadoop-mapreduce-client/hadoop-streami
ng.jar \
> -file mapper.py \
> -file reducer.py \
> -mapper mapper.py \
> -reducer reducer.py \
> -input /user/maria_dev/car_project/car_price_big.csv \
> -output /user/maria_dev/car_project/output_avg_brand
26/02/26 14:32:33 WARN streaming.StreamJob: -file option is deprecated, please use generic op
tion -files instead.
packageJobJar: [mapper.py, reducer.py] [/usr/hdp/2.6.5.0-292/hadoop-mapreduce/hadoop-streamin
g-2.7.3.2.6.5.0-292.jar] /tmp/streamjob1765025456034898948.jar tmpDir=null
26/02/26 14:32:35 INFO client.RMProxy: Connecting to ResourceManager at sandbox-hdp.hortonwor
ks.com/172.18.0.2:8032
26/02/26 14:32:35 INFO client.AHSPProxy: Connecting to Application History server at sandbox-h
dp.hortonworks.com/172.18.0.2:10200
26/02/26 14:32:35 INFO client.RMProxy: Connecting to ResourceManager at sandbox-hdp.hortonwor
ks.com/172.18.0.2:8032
26/02/26 14:32:35 INFO client.AHSPProxy: Connecting to Application History server at sandbox-h
dp.hortonworks.com/172.18.0.2:10200
26/02/26 14:32:37 INFO mapred.FileInputFormat: Total input paths to process : 1
26/02/26 14:32:37 INFO net.NetworkTopology: Adding a new node: /default-rack/172.18.0.2:50010
26/02/26 14:32:38 INFO mapreduce.JobSubmitter: number of splits:2
26/02/26 14:32:38 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1772114037925_0
004
26/02/26 14:32:39 INFO impl.YarnClientImpl: Submitted application application_1772114037925_0
004
26/02/26 14:32:39 INFO mapreduce.Job: The url to track the job: http://sandbox-hdp.hortonwork
s.com:8088/proxy/application_1772114037925_0004/
26/02/26 14:32:39 INFO mapreduce.Job: Running job: job_1772114037925_0004
26/02/26 14:32:47 INFO mapreduce.Job: Job job_1772114037925_0004 running in uber mode : false
26/02/26 14:32:47 INFO mapreduce.Job: map 0% reduce 0%
26/02/26 14:33:00 INFO mapreduce.Job: map 62% reduce 0%
26/02/26 14:33:01 INFO mapreduce.Job: map 100% reduce 0%
```

```

Total committed heap usage (bytes)=312999936
Shuffle Errors
  BAD_ID=0
  CONNECTION=0
  IO_ERROR=0
  WRONG_LENGTH=0
  WRONG_MAP=0
  WRONG_REDUCE=0
File Input Format Counters
  Bytes Read=160134156
File Output Format Counters
  Bytes Written=139
26/02/26 14:33:14 INFO streaming.StreamJob: Output directory: /user/aria_dev/car_project/out
put_avg_brand
[aria_dev@sandbox-hdp ~]$ ^C
[aria_dev@sandbox-hdp ~]$ hdfs dfs -cat /user/aria_dev/car_project/output_avg_brand/part-00
000
Audi      51953.4248098
BMW       54157.1143855
Ford      51593.2548127
Honda     52050.2839488
Mercedes  53191.090085
Tesla     53475.5474713
Toyota    52078.7282353
[aria_dev@sandbox-hdp ~]$ ^C
[aria_dev@sandbox-hdp ~]$ █

```

```

aria_dev@sandbox-hdp:~
#! /usr/bin/env python
import sys

for line in sys.stdin:
    line = line.strip()
    columns = line.split(",")

    # Skip header
    if columns[0] == "Car ID":
        continue

    if len(columns) > 8:
        brand = columns[1]
        price = columns[8]

        try:
            price = float(price)
            print "%s\t%s" % (brand, price)
        except:
            continue

```

🖱️ maria_dev@sandbox-hdp:~

```
#!/usr/bin/env python
import sys
```

```
current_brand = None
total = 0
count = 0
```

```
for line in sys.stdin:
    line = line.strip()
    parts = line.split("\t")
```

```
    if len(parts) != 2:
        continue
```

```
    brand, price = parts
```

```
    try:
        price = float(price)
    except:
        continue
```

```
    if current_brand == brand:
        total += price
        count += 1
```

```
    else:
        if current_brand is not None:
            print "%s\t%s" % (current_brand, total/float(count))
            current_brand = brand
            total = price
            count = 1
```

```
# Print last brand
```

```
if current_brand is not None:
    print "%s\t%s" % (current_brand, total/float(count))
```